

LAPORAN AKHIR PROJECT AOL CODE REENGINEERING (COMP6106001)



REFACTORING WEATHER APP ANDROID

Menangani God Class, Hardcoded Values, dan Dead Code

Oleh:

Kelas: LC83 / Kelompok 4

NIM dan Anggota

Kelompok

2702341070 - FELIXAN LAMHOT GERALDO

2702284432 - MUHAMMAD ZELDY NASUTION

2702338126 - BRYAN PAULO BHELE FE

2702341410 - ADAM HARISH ABDURRAHMAN

Program Studi Software Engineering – School of Computer Science

Universitas Bina Nusantara

Semester Ganjil 2025/2026

Daftar Isi

ABSTRAK.....	4
1. PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Identifikasi Code Smells.....	5
1.3 Alasan Code Smell.....	5
2. METODOLOGI.....	6
2.1 Persiapan Lingkungan.....	6
2.2 Teknik Refactoring yang Digunakan.....	6
2.2.1 Extract Class (Modularisasi).....	6
2.2.2 Extract Method.....	6
2.2.3 Remove Dead Code & Replace Constants.....	7
3. HASIL DAN ANALISIS.....	7
3.1 Perbandingan Struktur File.....	7
3.2 Analisis Pengurangan Kompleksitas.....	7
3.3 Validasi Fungsional.....	7
4. PEMBAHASAN (DISCUSSION).....	8
4.1 Tantangan (Challenges).....	8
4.2 Solusi.....	8
5. KESIMPULAN.....	8

6.	REFERENSI.....	9
7.	LAMPIRAN.....	9
7.1	Pembagian Tugas.....	9
7.2	Code Before Refactoring.....	9
7.3	Code After Refactoring.....	12

ABSTRAK

Pemeliharaan dan pengembangan perangkat lunak mobile kerap terhambat ketika Activity utama menanggung terlalu banyak tanggung jawab, sebuah pola yang dikenal sebagai God Class. Dokumen ini menyajikan tahapan perbaikan kode (reengineering) pada aplikasi **Weather App** yang dibangun untuk platform Android. Analisis awal kode sumber mengungkap HomeActivity sebagai titik sentral di mana logika UI, network, dan layanan lokasi dikonsolidasikan, melanggar prinsip pemisahan tanggung jawab. Selain itu, ditemukan data yang disematkan (hardcoded) yang berpotensi menimbulkan masalah fleksibilitas. Proses kerja ini mengadopsi langkah-langkah penemuan *code smell*, diikuti dengan implementasi teknik-teknik perombakan struktur kode (refactoring). Teknik yang krusial adalah Ekstraksi Kelas dan Ekstraksi Metode, serta pembersihan kode usang (*Dead Code removal*). Dampak positif dari refactoring ini adalah organisasi kode yang sangat modular dan peningkatan signifikan dalam keterbacaan. Struktur baru kini berjalan sesuai dengan Single Responsibility Principle (SRP), dengan fungsionalitas aplikasi yang tidak terpengaruh.

1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi Android, seringkali pengembang menempatkan seluruh logika bisnis dan manajemen data di dalam Activity. Seiring bertambahnya fitur, kelas ini menjadi sangat besar dan sulit dipelihara. Hal ini memunculkan "Code Smells", indikator kelemahan desain yang dapat menyebabkan technical debt.

Kelompok kami memilih studi kasus aplikasi **Weather App** yang memiliki indikasi kuat God Class dan Hardcoded Values. Kode asli menunjukkan struktur yang mencampurkan logika networking (Volley) dan manipulasi UI dalam satu file, sehingga sangat relevan untuk dilakukan proses refactoring.

1.2 Identifikasi Code Smells

Berdasarkan analisis awal menggunakan referensi materi kuliah, ditemukan beberapa code smells dominan:

1. The Bloater - God Class / Large Class:
Kelas HomeActivity.java memiliki tanggung jawab yang terlalu besar (UI, Permission, GPS Location, API Calls).
2. The Bloater - Long Method:
Method onCreate dan getTodayWeatherInfo terlalu panjang dan melakukan banyak tugas yang tidak saling berkaitan secara langsung.
3. Hardcoded Values:
Kelas Toaster.java menggunakan string warna hex ("454B54") secara langsung di dalam kode, bukan menggunakan resources Android (colors.xml), menyulitkan perubahan tema.
4. The Dispensable - Dead Code / Lazy Class:
Kelas CityFinder.java hanya berisi metode statis yang jarang digunakan dan kurang kohesif sebagai kelas yang berdiri sendiri.
5. Improper Instantiation:
Kelas URL.java diinstansiasi (new URL()) hanya untuk memanggil metode yang seharusnya bersifat statis.

1.3 Alasan Code Smell

Code smells tersebut muncul karena:

1. Pelanggaran Single Responsibility Principle (SRP):
Pengembang awal menggabungkan logika pengambilan data (Model) dan tampilan (View) dalam satu Activity, padahal HomeActivity seharusnya hanya bertanggung jawab menampilkan data.
2. Kesulitan Pengujian (Testing):
Logika bisnis yang bercampur dengan logika UI membuat unit testing sulit dilakukan.
3. Redundansi:
Penggunaan nilai hardcoded menyulitkan pemeliharaan jangka panjang.

2. METODOLOGI

Proses reengineering/refactoring dilakukan dengan langkah-langkah berikut:

2.1 Persiapan Lingkungan

Kami menggunakan **Git** sebagai Version Control System untuk melacak perubahan sebelum dan sesudah refactoring. Pengembangan dilakukan menggunakan IDE **Android Studio**. Setiap langkah refactoring dilakukan secara bertahap untuk memastikan aplikasi tetap berjalan di setiap tahapnya.

2.2 Teknik Refactoring yang Digunakan

2.2.1 Extract Class (Modularisasi)

Teknik ini digunakan untuk memecah God Class `HomeActivity`. Kami memisahkan tanggung jawab ke dalam kelas-kelas baru:

- **`WeatherRepository.java`**: Menangani seluruh logika jaringan (API Calls).
- **`LocationManager.java`**: Menangani izin lokasi dan pengambilan koordinat GPS (memindahkan logika dari `HomeActivity` dan `CityFinder`).
- **`HomeActivity.java`**: Sekarang hanya fokus pada inisialisasi UI dan menampilkan data.

2.2.2 Extract Method

Teknik ini diterapkan pada method `onCreate` di `HomeActivity`.

Logika inisialisasi dipecah menjadi metode privat yang deskriptif:

- `initHelpers()`
- `setupUI()`
- `setupListeners()`.

2.2.3 Remove Dead Code & Replace Constants

- Menghapus kelas `CityFinder.java` karena fungsinya digabungkan ke `LocationManager` yang lebih kohesif.
- Mengubah metode di `URL.java` menjadi `static` agar dapat diakses tanpa instansiasi objek.
- Mengganti string warna hardcoded di `Toaster.java` dengan referensi `R.color`.

3. HASIL DAN ANALISIS

3.1 Perbandingan Struktur File

Sebelum refactoring, proyek sangat bergantung pada satu file besar. Setelah refactoring, proyek menjadi lebih terdistribusi:

Before:

- `HomeActivity.java` (~331 LOC, mencakup semua logika).
- `CityFinder.java`.

After:

- `HomeActivity.java` (Fokus UI).
- `WeatherRepository.java` (Logic Network).
- `LocationManager.java` (Logic GPS).

3.2 Analisis Pengurangan Kompleksitas

Refactoring berhasil mengurangi kompleksitas dan meningkatkan kohesi kelas.

Tabel Perbandingan Metrik

Metrik	Sebelum Refactoring	Sesudah Refactoring
Ukuran HomeActivity	331 Baris Kode	Berkurang signifikan
Tanggung Jawab	UI, Network, Location, Permission	UI Only
Dependency	Tightly Coupled	Loosely Coupled
Keterbacaan	Rendah	Tinggi (Nama method deskriptif)
Fleksibilitas	Sulit mengganti library network	Mudah (Logic terisolasi di Repository)

3.3 Validasi Fungsional

Setelah dilakukan perubahan struktur, pengujian manual (Black Box Testing) dilakukan. Fitur utama seperti deteksi lokasi otomatis, pengambilan data cuaca hari ini, dan tampilan UI dipastikan tetap berjalan normal tanpa crash, membuktikan bahwa perilaku eksternal program tidak berubah.

4. PEMBAHASAN (DISCUSSION)

4.1 Tantangan (Challenges)

Tantangan terbesar adalah **memisahkan logika Asynchronous** (seperti request jaringan Volley dan callback lokasi) dari Activity. Activity kehilangan konteks untuk memperbarui UI saat data selesai dimuat karena proses jaringan berjalan di latar belakang.

4.2 Solusi

Solusi yang diterapkan adalah menggunakan pola **Callback Interface** (`OnWeatherDataListener`, `OnLocationListener`) di dalam Repository dan Manager. HomeActivity mengimplementasikan interface ini, sehingga tetap dapat menerima hasil data secara asinkron tanpa harus mengetahui detail implementasi pengambilan datanya.

5. KESIMPULAN

Berdasarkan proses code reengineering yang telah dilakukan, dapat disimpulkan bahwa:

1. Migrasi aplikasi dari **struktur terpusat** (*God Class*) menuju tata letak yang lebih modular telah diselesaikan dengan sukses.
2. Penerapan teknik **Extract Class** dan **Extract Method** berhasil memisahkan concern antara UI, Data, dan Lokasi sesuai prinsip Separation of Concerns.
3. Kode menjadi lebih mudah dibaca (readable) dan lebih siap untuk pengembangan fitur masa depan karena isolasi logika yang baik.

6. REFERENSI

1. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code*.
2. dev-aniketj. (2023). *WeatherApp-Android*. GitHub Repository. Diakses dari: <https://github.com/dev-aniketj/WeatherApp-Android>
3. Binus University. (2024). *Course Materials: Code Reengineering (COMP6106001)*.

7. LAMPIRAN (APPENDIX)

7.1 Pembagian Tugas (Division of Tasks)

Nama Anggota	NIM	Deskripsi Tugas
Adam Harish Abdurrahman	2702341410	Analisis Code Smells (God Class & Hardcoded Values), Penyusunan Bab 1-3.
Felixan Lamhot Geraldo	2702341070	Refactoring Modul Network (Extract Class: WeatherRepository) & Implementasi Callback.
Bryan Paulo Bhele Fe	2702338126	Refactoring Modul Location (Extract Class: LocationManager) & Remove Dead Code.
Muhammad Zeldy Nasution	2702284432	Finalisasi Laporan, Validasi Fungsional (Testing), dan Penyusunan Bab 4-5.

7.2 Code Before Refactoring (Cuplikan)

Link Repository Asli :

<https://github.com/dev-aniketj/WeatherApp-Android/blob/master/app/src/main/java/com/aniketjain/weatherapp/HomeActivity.java>

HomeActivity.java

#Location Network Section#

```
`private void getDataUsingNetwork() {
    FusedLocationProviderClient client = LocationServices.getFusedLocationProviderClient(this);
    //check permission
    if (ActivityCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) !=
PackageManager.PERMISSION_GRANTED && ActivityCompat.checkSelfPermission(this,
    Manifest.permission.ACCESS_COARSE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) {
        ActivityCompat.requestPermissions(this, new
String[] {Manifest.permission.ACCESS_FINE_LOCATION,
Manifest.permission.ACCESS_COARSE_LOCATION}, PERMISSION_CODE);
    } else {
        client.getLastLocation().addOnSuccessListener(location -> {
            setLongitudeLatitude(location);
            city = getCityNameUsingNetwork(this, location);
            getTodayWeatherInfo(city);
        });
    }
}`
```

#Weather Info Today#

```
,
@SuppressLint("DefaultLocale")
private void getTodayWeatherInfo(String name) {
    ...
    URL url = new URL();
    RequestQueue requestQueue = Volley.newRequestQueue(this);
    JsonObjectRequest request = new JsonObjectRequest(..., response -> { ... });
    requestQueue.add(request);
    ...
},
```

7.3 Code After Refactoring (Cuplikan)

Perbaikan / Refactoring yang telah dilakukan adalah separasi kelas yang bisa dipisahkan ke kelas yang lain

Link Repository Refactor:

<https://github.com/AdamHarish25/AOL/blob/main/weatherApp/WeatherApp-Android/app/src/main/java/com/aniketjain/weatherapp/HomeActivity.java>

HomeActivity.java

#Location Network Section#

```
`private void getDataUsingNetwork() {  
    locationManager.getLastLocation(this, new LocationManager.OnLocationListener() {  
        @Override  
        public void onLocationFound(Location location) {  
            // Logika UI saat lokasi ditemukan  
            LocationCord.lat = String.valueOf(location.getLatitude());  
            city = locationManager.getCityName(location);  
            getTodayWeatherInfo(city);  
        }  
        // ... handling error dan permission ...  
    });  
}`
```

#Weather Info Today#

```
`private void getTodayWeatherInfo(String name) {  
    weatherRepository.getWeatherData(LocationCord.lat, LocationCord.lon,  
    new WeatherRepository.OnWeatherDataListener() {  
        @Override  
        public void onSuccess(JSONObject response) {  
            // Update UI dengan data JSON yang diterima  
            update_time = response.getJSONObject("current").getLong("dt");  
            // ... parsing data lainnya ...  
            updateUI();  
        }  
  
        @Override  
        public void onError(Exception e) {  
            Toaster.errorToast(HomeActivity.this, "Error fetching weather data");  
        }  
    });  
}`
```

WeatherRepository.java

,

```
public class WeatherRepository {  
    // ... setup RequestQueue ...
```

```
    public void getWeatherData(String lat, String lon, final OnWeatherDataListener listener) {  
        String link = "https://api.openweathermap.org/data/2.5/onecall?..." + ...;
```

```
        JsonObjectRequest jsonObjectRequest = new JsonObjectRequest(Request.Method.GET, link,  
null, response -> {  
            listener.onSuccess(response); // Callback sukses  
        }, listener::onError); // Callback error
```

```
        requestQueue.add(jsonObjectRequest);  
    }  
}
```

,

LocationManager.java

```
,  
  
public class LocationManager {  
    // ... inialisasi FusedLocationProviderClient ...  
  
    public void getLastLocation(Activity activity, final OnLocationListener listener) {  
        // Cek permission terpusat di sini  
        if (ActivityCompat.checkSelfPermission(context, Manifest.permission.ACCESS_FINE_LOCATION) !=  
PackageManager.PERMISSION_GRANTED ...) {  
            listener.onPermissionNeeded();  
            return;  
        }  
  
        // Ambil lokasi terakhir  
        fusedLocationClient.getLastLocation().addOnSuccessListener(activity, location -> {  
            if (location != null) {  
                listener.onLocationFound(location);  
            } else {  
                listener.onLocationError(new Exception("Location is null"));  
            }  
        });  
    }  
  
    public String getCityName(Location location) {  
        // Logika Geocoder untuk mengubah lat/lon menjadi nama kota  
        // ...  
    }  
}  
,
```